

Agent Skills sob Análise

É importante? É aplicável aqui? Este projeto já tem estrutura própria?

Agosto de 2026

Sumário

1. O que é, de verdade (correção de origem)	1
2. É importante/útil, de forma genérica? Sim, com ressalva	1
3. Aplicável a ESTE projeto? Depende de qual metade do projeto	2
3.1 Domínio A — a Fábrica de Materiais de Comunicação em si	2
3.2 Domínio B — a engenharia por trás do sistema (painel/, scripts/, testes)	2
4. Tensão real com a prioridade #1 do projeto	3
5. Meu veredito final	3
6. Fontes	4

1. O que é, de verdade (correção de origem)

O MANUAL_AGENT_SKILLS.md anterior dizia que as 23 skills vieram “via /reload-plugins”. Isso estava impreciso. Rastreando skills-lock.json (raiz do repo), a origem real é:

- **Repositório:** addyosmani/agent-skills — “Production-grade engineering skills for AI coding agents.”
- **Autor:** Addy Osmani (ex-Google/Chrome, figura conhecida em engenharia web e ferramentas de IA para código).
- **Licença:** MIT. **86.947 estrelas.** Criado fev/2026, push mais recente em 11/ago/2026 — ativo, popular, não é projeto obscuro ou abandonado.
- **Mecanismo de instalação:** um fetch direto de skills individuais do GitHub, fixado por hash (skills-lock.json), **não** o plugin i-have-adhd (esse é outra coisa: um estilizador de output pra leitor com TDAH, de outro autor, que só coincidiu de recarregar no mesmo comando).

Ou seja: a procedência é sólida. Isso não significa que o conteúdo seja necessário *aqui* — são perguntas diferentes.

2. É importante/útil, de forma genérica? Sim, com ressalva

O catálogo (interview-me, spec-driven-development, TDD, code-review, observability, etc.) é uma destilação razoável de boas práticas de engenharia de software mainstream, bem organizada por fase (Definir → Planejar → Construir → Verificar → Revisar → Entregar). Não é *snake oil*: os “Comportamentos centrais” (expor suposições, não ser sim-máquina, disciplina de escopo, verificar em vez de assumir) são princípios genuinamente bons.

Prova real, não hipotética: interview-me foi usado nesta própria sessão para decidir o redesign do painel — e mudou o resultado. Sem a entrevista, eu teria construído um wizard sequencial (a resposta convencional ao pedido original). A entrevista revelou que o problema real era visual, não de

navegação, e que um wizard pioraria a visibilidade que o operador já valoriza. Isso é valor demonstrado, não teórico.

3. Aplicável a ESTE projeto? Depende de qual metade do projeto

Este repositório tem **dois domínios de trabalho muito diferentes**, e a resposta muda completamente entre eles.

3.1 Domínio A — a Fábrica de Materiais de Comunicação em si

Geração de PDF/landing/apresentação/artes/kits/textos via analista-insumos → diretor-de-arte → redator-* → compilador-* → revisor-marca, regida por AGENTS.md/SPEC_COMANDOS.md.

Quase nada do Agent Skills se aplica aqui — e não é por falta do framework, é porque este projeto já resolveu o mesmo problema de forma mais específica e mais madura:

- spec-driven-development genérico (“escreva requisitos antes de codar”) já existe aqui, só que **muito mais específico e testado**: SPEC_COMANDOS.md + config_projeto.json/brief_criativo.json como contrato formal por projeto.
- test-driven-development não se aplica a redigir uma apostila ou uma landing page — a “prova” de um material de comunicação é scripts/validar-*.py --estrito, não teste unitário no sentido clássico.
- REGRA 6 (“nunca inventar claim, sempre citar fonte do texto-base”) já é uma forma de “verifique, não assuma” — só que domain-specific e mais rígida do que o comportamento genérico do framework.
- code-review-and-quality não tem sentido pra copy publicitário — quem cumpre esse papel aqui é revisor-marca (fidelidade à fonte + fidelidade à marca, com evidência de script).

Veredito domínio A: framework genérico é irrelevante aqui. O projeto não está “sem estrutura” esperando o Agent Skills chegar — o oposto: tem uma estrutura *mais específica e mais adequada ao domínio* do que qualquer skill genérica de SDLC poderia oferecer.

3.2 Domínio B — a engenharia por trás do sistema (painel/, scripts/, testes)

Aqui sim é código de verdade (Python, FastAPI, HTML/JS, pytest) — o tipo de trabalho para o qual o Agent Skills foi desenhado. Mas mesmo aqui, quando eu cruzo cada skill nova com o que **já existe neste repo**, a maioria esbarra em algo equivalente:

- Debugar: debug-issue/diagnosing-bugs/systematic-debugging (já existiam, um deles *graph-powered* via code-review-graph) vs. debugging-and-error-recovery (novo) — redundante, e a versão antiga é mais barata em tokens dentro deste repo.
- Revisar código: code-review/review-changes (já existiam) vs. code-review-and-quality (novo) — redundante.
- TDD: tdd (já existia) vs. test-driven-development (novo) — **mesmo processo, literalmente duplicado**, nomes diferentes.
- Simplificar: simplify (já existia) vs. code-simplification (novo) — redundante.
- Refatorar: refactor-safely (já existia, graph-powered) — sem contrapartida direta nova, mas já coberto.
- Segurança: security-review (já existia) vs. security-and-hardening (novo) — redundante.

- Planejar: `writing-plans/executing-plans/subagent-driven-development` (já existiam) vs. `planning-and-task-breakdown/incremental-implementation` (novos) — sobreposição forte.
- Verificar antes de declarar pronto: `verification-before-completion` (já existia) — o framework novo espalha isso como passo de cada skill individual, mesma ideia.
- Economia de contexto: `lean-ctx` (já existia, é a **prioridade #1** deste projeto) vs. `context-engineering` (novo, genérico) — o antigo é mais específico e mais alinhado ao que o `CLAUDE.md` já exige.

O que é genuinamente novo e sem equivalente aqui (poucas, mas reais): `frontend-ui-engineering` (relevante pro redesign visual do painel que está pendente), `api-and-interface-design` (relevante se a API do painel crescer), `source-driven-development` (foi, na prática, o que fiz manualmente ao pesquisar a sintaxe real dos 5 harnesses novos antes de codar os adaptadores), `doubt-driven-development` (complementar, não redundante, ao `fable-judge` já existente — este último é uma auditoria *depois* de declarar pronto; aquele é contra-exame *durante* a construção).

Veredito domínio B: parcialmente aplicável — um punhado de skills preenche lacuna real (UI, design de API, verificação contra doc externa), o resto é redundante com algo que já existia e, no caso do TDD/debug/review, a versão antiga costuma ser mais barata em tokens por já conhecer o grafo do repo.

4. Tensão real com a prioridade #1 do projeto

O `CLAUDE.md` deste repo declara, em letras maiúsculas, que **Economia Severa de Tokens** é prioridade máxima: respostas telegráficas, compressão de log, delegação a subagentes, grep antes de read. A cadeia completa que o Agent Skills recomenda para uma feature nova tem **16 passos** (`interview-me` → `idea-refine` → `spec-driven-development` → `planning-and-task-breakdown` → `context-engineering` → `source-driven-development` → `incremental-implementation` → `observability-and-instrumentation` → `doubt-driven-development` → `test-driven-development` → `code-review-and-quality` → `code-simplification` → `git-workflow-and-versioning` → `documentation-and-adrs` → `deprecation-and-migration` → `shipping-and-launch`).

Rodar essa cadeia inteira para um ajuste típico no painel (um botão, uma correção de bug) seria **desproporcional e caro em tokens** — na contramão direta da prioridade #0 deste projeto. O próprio framework reconhece isso (“nem toda tarefa precisa de todos os passos”), mas a tentação de segui-lo à risca por ser “o processo certo” é um risco real de custo, não hipotético.

5. Meu veredito final

Não é lixo, não é essencial — é uma ferramenta de prateleira genérica que este projeto, na maior parte, já não precisa, porque já construiu (ao longo de meses, não desta sessão) uma estrutura mais específica pro domínio que importa aqui (materiais de comunicação) e uma estrutura equivalente ou melhor pro domínio de engenharia (grafo de código, economia de tokens, `fable-judge`, `verification-before-completion`).

Não recomendo desinstalar — custo zero de manter parado, e as poucas skills genuinamente novas (`frontend-ui-engineering`, `source-driven-development`, `doubt-driven-development`, `interview-me`) têm valor real e já provado nesta sessão. **Recomendo não seguir a cadeia completa por padrão**

— usar seletivamente essas 4, e preferir sempre a versão já existente e graph-powered quando ela cobrir a mesma necessidade (ver seção 3.2 e a tabela de sobreposições do `MANUAL_AGENT_SKILLS.md`).

6. Fontes

- `addyosmani/agent-skills` (GitHub)
- `skills-lock.json` (raiz deste repo) — evidência da origem real das 23 skills
- `CLAUDE.md` deste repo — prioridade de economia de tokens citada na seção 4